

JSON Batch Processing

Absolutely. Since you want **batch processing**, let's make this a proper JSON practice dataset and then run through the major JSON transformations you should know in PySpark.

We'll practice:

1. Read JSON
2. Inspect schema
3. Select columns
4. Select nested fields
5. Flatten nested JSON
6. Rename columns
7. Cast data types
8. Handle nulls
9. Filter / where
10. Add columns
11. String transformations
12. Date/timestamp transformations
13. Numeric transformations
14. Conditional transformations
15. Array transformations
16. Explode arrays
17. Struct transformations
18. Aggregations
19. GroupBy
20. Window functions
21. Sort
22. Drop duplicates
23. Drop columns
24. JSON → JSON
25. JSON → Parquet

1. Create a larger JSON file

Save this as:

```
/data/json/orders.json
```

You can use multiple JSON objects, one per line (JSON Lines format), which is ideal for Spark.

```
{"order_id":"ORD1001","timestamp":"2026-08-01T10:15:00Z","customer":{"customer_id":501,"name":"John Doe","email":"john@example.com","address":{"city":"Toronto","country":"Canada"}}, "product":{"name":"Laptop","category":"Electronics","price":1200.50},"qua
```

```
ntity":2,"status":"completed","payment_method":"credit_card","tags":["electronics","computer","sale"]}  
{  
  "order_id":"ORD1002","timestamp":"2026-08-01T11:20:00Z",  
  "customer":{"customer_id":502,"name":"Alice Smith",  
    "email":"alice@example.com",  
    "address":{"city":"New York",  
      "country":"USA"}},  
  "product":{"name":"Phone",  
    "category":"Electronics",  
    "price":850.00},  
  "quantity":1,  
  "status":"completed",  
  "payment_method":"paypal",  
  "tags":["electronics","mobile"]  
}  
{  
  "order_id":"ORD1003","timestamp":"2026-08-01T12:05:00Z",  
  "customer":{"customer_id":503,"name":"Bob Johnson",  
    "email":"bob@example.com",  
    "address":{"city":"London",  
      "country":"UK"}},  
  "product":{"name":"Headphones",  
    "category":"Electronics",  
    "price":150.75},  
  "quantity":3,  
  "status":"pending",  
  "payment_method":"credit_card",  
  "tags":["audio","electronics"]  
}  
{  
  "order_id":"ORD1004","timestamp":"2026-08-01T13:10:00Z",  
  "customer":{"customer_id":504,"name":"Emma Wilson",  
    "email":"emma@example.com",  
    "address":{"city":"Paris",  
      "country":"France"}},  
  "product":{"name":"Desk",  
    "category":"Furniture",  
    "price":300.00},  
  "quantity":1,  
  "status":"completed",  
  "payment_method":"bank_transfer",  
  "tags":["office","furniture"]  
}  
{  
  "order_id":"ORD1005","timestamp":"2026-08-01T14:25:00Z",  
  "customer":{"customer_id":505,"name":"Michael Brown",  
    "email":"michael@example.com",  
    "address":{"city":"Berlin",  
      "country":"Germany"}},  
  "product":{"name":"Chair",  
    "category":"Furniture",  
    "price":180.25},  
  "quantity":4,  
  "status":"cancelled",  
  "payment_method":"credit_card",  
  "tags":["office","furniture"]  
}  
{  
  "order_id":"ORD1006","timestamp":"2026-08-02T09:15:00Z",  
  "customer":{"customer_id":506,"name":"Sophia Davis",  
    "email":"sophia@example.com",  
    "address":{"city":"Dubai",  
      "country":"UAE"}},  
  "product":{"name":"Monitor",  
    "category":"Electronics",  
    "price":450.00},  
  "quantity":2,  
  "status":"completed",  
  "payment_method":"paypal",  
  "tags":["computer","office","electronics"]  
}  
{  
  "order_id":"ORD1007","timestamp":"2026-08-02T10:30:00Z",  
  "customer":{"customer_id":507,"name":"James Miller",  
    "email":"james@example.com",  
    "address":{"city":"Sydney",  
      "country":"Australia"}},  
  "product":{"name":"Keyboard",  
    "category":"Electronics",  
    "price":90.50},  
  "quantity":5,  
  "status":"completed",  
  "payment_method":"credit_card",  
  "tags":["computer","accessories"]  
}  
{  
  "order_id":"ORD1008","timestamp":"2026-08-02T11:45:00Z",  
  "customer":{"customer_id":508,"name":"Olivia Taylor",  
    "email":"olivia@example.com",  
    "address":{"city":"Madrid",  
      "country":"Spain"}},  
  "product":{"name":"Table",  
    "category":"Furniture",  
    "price":550.00},  
  "quantity":1,  
  "status":"pending",  
  "payment_method":"bank_transfer",  
  "tags":["home","furniture"]  
}  
{  
  "order_id":"ORD1009","timestamp":"2026-08-02T13:00:00Z",  
  "customer":{"customer_id":509,"name":"William Anderson",  
    "email":"william@example.com",  
    "address":{"city":"Rome",  
      "country":"Italy"}},  
  "product":{"name":"Camera",  
    "category":"Electronics",  
    "price":980.00},  
  "quantity":1,  
  "status":"completed",  
  "payment_method":"credit_card",  
  "tags":["camera","photography","electronics"]  
}  
{  
  "order_id":"ORD1010","timestamp":"2026-08-02T14:20:00Z",  
  "customer":{"customer_id":510,"name":"Ava Thomas",  
    "email":"ava@example.com",  
    "address":{"city":"Cairo",  
      "country":"Egypt"}},  
  "product":{"name":"Printer",  
    "category":"Electronics",  
    "price":275.00},  
  "quantity":2,  
  "status":"completed",  
  "payment_method":"cash",  
  "tags":["office","printer"]  
}
```

```
{
  "order_id": "ORD1011",
  "timestamp": "2026-08-03T09:10:00Z",
  "customer": {
    "customer_id": 511,
    "name": "Daniel Jackson",
    "email": "daniel@example.com",
    "address": {
      "city": "Chicago",
      "country": "USA"
    }
  },
  "product": {
    "name": "Tablet",
    "category": "Electronics",
    "price": 620.00
  },
  "quantity": 2,
  "status": "pending",
  "payment_method": "paypal",
  "tags": ["mobile", "electronics"]
},
{
  "order_id": "ORD1012",
  "timestamp": "2026-08-03T10:40:00Z",
  "customer": {
    "customer_id": 512,
    "name": "Isabella White",
    "email": "isabella@example.com",
    "address": {
      "city": "Vancouver",
      "country": "Canada"
    }
  },
  "product": {
    "name": "Sofa",
    "category": "Furniture",
    "price": 1100.00
  },
  "quantity": 1,
  "status": "completed",
  "payment_method": "credit_card",
  "tags": ["home", "furniture"]
},
{
  "order_id": "ORD1013",
  "timestamp": "2026-08-03T12:00:00Z",
  "customer": {
    "customer_id": 513,
    "name": "Matthew Harris",
    "email": "matthew@example.com",
    "address": {
      "city": "Amsterdam",
      "country": "Netherlands"
    }
  },
  "product": {
    "name": "Mouse",
    "category": "Electronics",
    "price": 45.99
  },
  "quantity": 6,
  "status": "completed",
  "payment_method": "credit_card",
  "tags": ["computer", "accessories"]
},
{
  "order_id": "ORD1014",
  "timestamp": "2026-08-03T13:30:00Z",
  "customer": {
    "customer_id": 514,
    "name": "Mia Martin",
    "email": "mia@example.com",
    "address": {
      "city": "Lisbon",
      "country": "Portugal"
    }
  },
  "product": {
    "name": "Bookshelf",
    "category": "Furniture",
    "price": 275.50
  },
  "quantity": 2,
  "status": "cancelled",
  "payment_method": "cash",
  "tags": ["home", "furniture"]
},
{
  "order_id": "ORD1015",
  "timestamp": "2026-08-03T15:00:00Z",
  "customer": {
    "customer_id": 515,
    "name": "David Thompson",
    "email": "david@example.com",
    "address": {
      "city": "Boston",
      "country": "USA"
    }
  },
  "product": {
    "name": "Smartwatch",
    "category": "Electronics",
    "price": 320.00
  },
  "quantity": 3,
  "status": "completed",
  "payment_method": "paypal",
  "tags": ["wearable", "electronics"]
},
{
  "order_id": "ORD1016",
  "timestamp": "2026-08-04T09:45:00Z",
  "customer": {
    "customer_id": 516,
    "name": "Charlotte Garcia",
    "email": "charlotte@example.com",
    "address": {
      "city": "Mexico City",
      "country": "Mexico"
    }
  },
  "product": {
    "name": "Lamp",
    "category": "Home",
    "price": 75.25
  },
  "quantity": 4,
  "status": "completed",
  "payment_method": "credit_card",
  "tags": ["home", "lighting"]
},
{
  "order_id": "ORD1017",
  "timestamp": "2026-08-04T11:15:00Z",
  "customer": {
    "customer_id": 517,
    "name": "Joseph Martinez",
    "email": "joseph@example.com",
    "address": {
      "city": "Los Angeles",
      "country": "USA"
    }
  },
  "product": {
    "name": "TV",
    "category": "Electronics",
    "price": 1450.00
  },
  "quantity": 1,
  "status": "completed",
  "payment_method": "credit_card",
  "tags": ["entertainment", "electronics"]
},
{
  "order_id": "ORD1018",
  "timestamp": "2026-08-04T12:30:00Z",
  "customer": {
    "customer_id": 518,
    "name": "Amelia Robinson",
    "email": "amelia@example.com",
    "address": {
      "city": "Manchester",
      "country": "UK"
    }
  },
  "product": {
    "name": "Bed",
    "category": "Furniture",
    "price": 900.00
  },
  "quantity": 1,
  "status": "pending",
  "payment_method": "bank_transfer",
  "tags": ["bedroom", "furniture"]
},
{
  "order_id": "ORD1019",
  "timestamp": "2026-08-04T14:10:00Z",
  "customer": {
    "customer_id": 519,
    "name": "Christopher Clark",
    "email": "christopher@example.com",
    "address": {
      "city": "Singapore",
      "country": "Singapore"
    }
  },
  "product": {
    "name": "Gaming PC",
    "category": "Electronics",
    "price": 2100.00
  },
  "quantity": 1,
  "status": "completed",
  "payment_method": "credit_card",
  "tags": ["gaming", "computer", "electronics"]
},
{
  "order_id": "ORD1020",
  "timestamp": "2026-08-04T16:00:00Z",
  "customer": {
    "customer_id": 520,
    "name": "Harper Rodriguez",
    "email": "harper@example.com",
    "address": {
      "city": "S\u00e3o"
    }
  }
}
```

```
Paulo", "country": "Brazil"}}, "product": {"name": "Coffee  
Table", "category": "Furniture", "price": 350.00}, "quantity": 2, "status": "complete  
d", "payment_method": "cash", "tags": ["home", "furniture"]}
```

2. Read the JSON with PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window

spark = (
    SparkSession.builder
        .appName("JSON Batch Processing Practice")
        .getOrCreate()
)

df = spark.read.json("hdfs://localhost:9000/data/json/orders.json")

df.printSchema()

df.show(truncate=False)
```

You should see nested structures similar to:

```
root
|-- customer: struct
|   |-- address: struct
|   |-- customer_id: long
|   |-- email: string
|   |-- name: string
|-- order_id: string
|-- payment_method: string
|-- product: struct
|-- quantity: long
|-- status: string
|-- tags: array
|-- timestamp: string
```

3. Select columns

```
df.select(
    "order_id",
    "quantity",
    "status"
).show()
```

Select nested columns:

```
df.select(
    "order_id",
    "customer.name",
    "customer.address.city",
    "product.name",
    "product.price"
).show()
```

4. Flatten the JSON

This is **very important** in real-world data engineering.

```
flat_df = df.select(
    "order_id",
    "timestamp",
    col("customer.customer_id").alias("customer_id"),
    col("customer.name").alias("customer_name"),
    col("customer.email").alias("customer_email"),
    col("customer.address.city").alias("city"),
    col("customer.address.country").alias("country"),
    col("product.name").alias("product"),
    col("product.category").alias("category"),
    col("product.price").alias("price"),
    "quantity",
    "status",
    "payment_method",
    "tags"
)

flat_df.show(truncate=False)
```

Now you have a much easier DataFrame to work with.

5. Calculate total price

```
df2 = flat_df.withColumn(
    "total_amount",
    col("price") * col("quantity")
)

df2.select(
    "order_id",
    "product",
    "price",
    "quantity",
    "total_amount"
).show()
```

6. Rename columns

```
df3 = df2.withColumnRenamed(  
    "total_amount",  
    "total_sales"  
)  
  
df3.show()
```

You can also rename multiple columns:

```
df3 = (  
    df2  
    .withColumnRenamed("product", "product_name")  
    .withColumnRenamed("price", "unit_price")  
    .withColumnRenamed("quantity", "quantity_sold")  
)
```

7. Cast data types

Convert timestamp:

```
df4 = df2.withColumn(  
    "timestamp",  
    to_timestamp("timestamp")  
)  
  
df4.printSchema()
```

Convert quantity:

```
df4 = df4.withColumn(  
    "quantity",  
    col("quantity").cast("integer")  
)
```

Convert price:

```
df4 = df4.withColumn(  
    "price",  
    col("price").cast("double")  
)
```

8. String transformations

Uppercase

```
df4.select(
    upper("customer_name").alias("customer_name_upper")
).show()
```

Lowercase

```
df4.select(
    lower("customer_email").alias("email_lower")
).show()
```

Trim

```
df4.withColumn(
    "customer_name",
    trim("customer_name")
)
```

Length

```
df4.select(
    "customer_name",
    length("customer_name").alias("name_length")
).show()
```

Substring

```
df4.select(
    substring("order_id", 4, 4).alias("order_number")
).show()
```

Replace

```
df4.select(
    regexp_replace(
        "customer_name",
        " ",
        "_"
    ).alias("name")
).show()
```

9. Filter / where

Completed orders:

```
completed_df = df4.filter(
    col("status") == "completed"
)

completed_df.show()
```

Multiple conditions:

```
df4.filter(
    (col("status") == "completed") &
    (col("total_amount") > 1000)
).show()
```

Using `where()`:

```
df4.where(
    col("category") == "Electronics"
).show()
```

10. Conditional transformation

Use `when()` and `otherwise()`.

```
df5 = df4.withColumn(
    "order_size",
    when(col("total_amount") >= 2000, "Very Large")
    .when(col("total_amount") >= 1000, "Large")
    .when(col("total_amount") >= 500, "Medium")
    .otherwise("Small")
)

df5.select(
    "order_id",
    "total_amount",
    "order_size"
).show()
```

This is extremely common in PySpark.

11. Date transformations

Extract date:

```
df6 = df5.withColumn(
    "order_date",
    to_date("timestamp")
)
```

Extract year:

```
df6 = df6.withColumn(
    "year",
```

```
        year("timestamp")
    )
```

Extract month:

```
df6 = df6.withColumn(
    "month",
    month("timestamp")
)
```

Extract day:

```
df6 = df6.withColumn(
    "day",
    day("timestamp")
)
```

Extract hour:

```
df6 = df6.withColumn(
    "hour",
    hour("timestamp")
)
```

Extract day of week:

```
df6 = df6.withColumn(
    "day_of_week",
    dayofweek("timestamp")
)
```

12. Numeric transformations

Absolute value:

```
df6.select(
    abs("price").alias("price")
).show()
```

Round:

```
df6.select(
    round("total_amount", 2).alias("rounded_total")
).show()
```

Ceiling:

```
df6.select(
    ceil("price")
)
```

```
).show()
```

Floor:

```
df6.select(  
    floor("price")  
) .show()
```

13. Handle NULL values

Find nulls:

```
df6.select(  
    *[  
        count(when(col(c).isNull(), c)).alias(c)  
        for c in df6.columns  
    ]  
) .show()
```

Drop null rows:

```
df6.dropna().show()
```

Fill nulls:

```
df6.fillna({  
    "quantity": 0,  
    "status": "unknown"  
})
```

14. Array transformations

Our JSON contains:

```
tags
```

Example:

```
["electronics", "computer", "sale"]
```

Get first element:

```
df6.select(  
    "order_id",  
    element_at("tags", 1).alias("first_tag")  
) .show()
```

Array size:

```
df6.select(
    "order_id",
    size("tags").alias("number_of_tags")
).show()
```

Check whether array contains something:

```
df6.filter(
    array_contains("tags", "electronics")
).show()
```

15. Explode the array

This is another **very important JSON transformation**.

```
exploded_df = df6.select(
    "order_id",
    "product",
    explode("tags").alias("tag")
)

exploded_df.show()
```

One row:

```
ORD1001 ["electronics","computer","sale"]
```

becomes:

```
ORD1001 electronics
ORD1001 computer
ORD1001 sale
```

You can also use:

```
explode_outer("tags")
```

when you want to preserve rows with null arrays.

16. Aggregations

Total sales:

```
df6.select(  
    sum("total_amount").alias("total_sales")  
).show()
```

Average order:

```
df6.select(  
    avg("total_amount").alias("average_order")  
).show()
```

Maximum:

```
df6.select(  
    max("total_amount").alias("maximum_order")  
).show()
```

Minimum:

```
df6.select(  
    min("total_amount").alias("minimum_order")  
).show()
```

Count:

```
df6.select(  
    count("*").alias("number_of_orders")  
).show()
```

17. GroupBy

Sales by category:

```
df6.groupBy("category").agg(  
    sum("total_amount").alias("total_sales"),  
    avg("total_amount").alias("average_sales"),  
    count("*").alias("number_of_orders")  
).show()
```

Sales by city:

```
df6.groupBy("city").agg(  
    sum("total_amount").alias("total_sales")  
).show()
```

Sales by payment method:

```
df6.groupBy("payment_method").agg(  
    count("*").alias("orders"),  
    sum("total_amount").alias("sales")  
)
```

```
).show()
```

18. GroupBy + multiple aggregations

```
category_summary = (  
    df6  
    .groupBy("category")  
    .agg(  
        count("*").alias("orders"),  
        sum("quantity").alias("items_sold"),  
        sum("total_amount").alias("total_sales"),  
        avg("total_amount").alias("average_order"),  
        max("total_amount").alias("largest_order"),  
        min("total_amount").alias("smallest_order")  
    )  
)  
  
category_summary.show()
```

19. HAVING-style filtering

In PySpark, aggregate first, then filter.

```
df6.groupBy("category").agg(  
    sum("total_amount").alias("total_sales")  
) .filter(  
    col("total_sales") > 3000  
) .show()
```

20. Sort

Ascending:

```
df6.orderBy(  
    "total_amount"  
) .show()
```

Descending:

```
df6.orderBy(  
    col("total_amount").desc()  
) .show()
```

Multiple columns:

```
df6.orderBy(  
    col("category").asc(),  
    col("total_amount").desc()  
).show()
```

21. Drop duplicates

```
df6.dropDuplicates().show()
```

Based on order ID:

```
df6.dropDuplicates(  
    ["order_id"]  
).show()
```

22. Drop columns

```
df6.drop(  
    "tags",  
    "customer_email"  
).show()
```

23. Window functions

This is important for your next level of PySpark.

```
window = (  
    Window  
    .partitionBy("category")  
    .orderBy(col("total_amount").desc())  
)
```

Rank orders:

```
ranked_df = df6.withColumn(  
    "rank",  
    row_number().over(window)  
)
```

```
ranked_df.select(  
    "category",  
    "product",  
    "total_amount",  
    "rank"  
) .show()
```

24. Rank within each category

```
ranked_df = df6.withColumn(  
    "rank",  
    dense_rank().over(window)  
)  
  
ranked_df.select(  
    "category",  
    "product",  
    "total_amount",  
    "rank"  
) .show()
```

25. Running total

```
window_running = (  
    Window  
    .partitionBy("category")  
    .orderBy("timestamp")  
    .rowsBetween(  
        Window.unboundedPreceding,  
        Window.currentRow  
    )  
)  
  
df7 = df6.withColumn(  
    "running_sales",  
    sum("total_amount").over(window_running)  
)  
  
df7.select(  
    "category",  
    "timestamp",  
    "total_amount",  
    "running_sales"  
) .show()
```

26. JSON → JSON

Save the transformed DataFrame as JSON:

```
df6.write \  
    .mode("overwrite") \  
    .json("hdfs://localhost:9000/data/output/orders_json")
```

Check it:

```
hdfs dfs -ls -R /data/output/orders_json
```

27. JSON → Parquet

This is a very common data-engineering workflow:

```
df6.write \
    .mode("overwrite") \
    .parquet("hdfs://localhost:9000/data/output/orders_parquet")
```

Check:

```
hdfs dfs -ls -R /data/output/orders_parquet
```

Read it back:

```
parquet_df = spark.read.parquet(
    "hdfs://localhost:9000/data/output/orders_parquet"
)

parquet_df.show()
```

28. Complete practice script

After practicing each transformation separately, you can combine them into one pipeline:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window

# =====
# 1. Spark Session
# =====

spark = (
    SparkSession.builder
        .appName("JSON Batch Processing Practice")
        .getOrCreate()
)

print(f"Spark version: {spark.version}")

# =====
# 2. Read JSON
# =====
```

```

df = spark.read.json(
    "hdfs://localhost:9000/data/json/orders.json"
)

print("Original Schema:")
df.printSchema()

# =====
# 3. Flatten nested JSON
# =====

df = df.select(
    "order_id",
    "timestamp",
    col("customer.customer_id").alias("customer_id"),
    col("customer.name").alias("customer_name"),
    col("customer.email").alias("customer_email"),
    col("customer.address.city").alias("city"),
    col("customer.address.country").alias("country"),
    col("product.name").alias("product"),
    col("product.category").alias("category"),
    col("product.price").alias("price"),
    "quantity",
    "status",
    "payment_method",
    "tags"
)

# =====
# 4. Cast columns
# =====

df = (
    df
    .withColumn("timestamp", to_timestamp("timestamp"))
    .withColumn("quantity", col("quantity").cast("integer"))
    .withColumn("price", col("price").cast("double"))
)

# =====
# 5. Calculate total amount
# =====

df = df.withColumn(
    "total_amount",
    col("price") * col("quantity")
)

# =====
# 6. Date transformations
# =====

df = (
    df
    .withColumn("order_date", to_date("timestamp"))
    .withColumn("year", year("timestamp"))

```

```

        .withColumn("month", month("timestamp"))
        .withColumn("day", day("timestamp"))
        .withColumn("hour", hour("timestamp"))
    )

# =====
# 7. String transformations
# =====

df = (
    df
    .withColumn("customer_name", trim("customer_name"))
    .withColumn("customer_name_upper", upper("customer_name"))
    .withColumn("email_lower", lower("customer_email"))
)

# =====
# 8. Conditional transformation
# =====

df = df.withColumn(
    "order_size",
    when(col("total_amount") >= 2000, "Very Large")
    .when(col("total_amount") >= 1000, "Large")
    .when(col("total_amount") >= 500, "Medium")
    .otherwise("Small")
)

# =====
# 9. Array transformations
# =====

df = (
    df
    .withColumn("tag_count", size("tags"))
    .withColumn("first_tag", element_at("tags", 1))
)

# =====
# 10. Filter
# =====

completed_df = df.filter(
    col("status") == "completed"
)

# =====
# 11. Aggregation
# =====

category_summary = (
    completed_df
    .groupBy("category")
    .agg(
        count("*").alias("orders"),
        sum("quantity").alias("items_sold"),
        sum("total_amount").alias("total_sales"),
    )
)

```

```

        avg("total_amount").alias("average_order"),
        max("total_amount").alias("largest_order"),
        min("total_amount").alias("smallest_order")
    )
    .orderBy(col("total_sales").desc())
)

# =====
# 12. Window function
# =====

window = (
    Window
    .partitionBy("category")
    .orderBy(col("total_amount").desc())
)

ranked_df = completed_df.withColumn(
    "rank",
    row_number().over(window)
)

# =====
# 13. Display
# =====

print("\n===== TRANSFORMED DATA =====")

df.select(
    "order_id",
    "customer_name",
    "city",
    "product",
    "category",
    "price",
    "quantity",
    "total_amount",
    "status",
    "order_size",
    "order_date",
    "tag_count"
).show(30, truncate=False)

print("\n===== CATEGORY SUMMARY =====")

category_summary.show(truncate=False)

print("\n===== TOP ORDERS PER CATEGORY =====")

ranked_df.filter(
    col("rank") <= 3
).select(
    "category",
    "product",
    "total_amount",
    "rank"
).show(truncate=False)

```

```

# =====
# 14. Save JSON
# =====

df.write \
    .mode("overwrite") \
    .json(
        "hdfs://localhost:9000/data/output/orders_json"
    )

# =====
# 15. Save Parquet
# =====

df.write \
    .mode("overwrite") \
    .parquet(
        "hdfs://localhost:9000/data/output/orders_parquet"
    )

print("\nData saved successfully.")

spark.stop()

```

Your practice progression

I recommend doing this in **three levels**:

Level 1 — Basic JSON

read.json → select → filter → withColumn → cast → rename → drop → sort

Level 2 — Nested JSON

struct → nested fields → flatten → arrays → explode → string/date/numeric transformations → groupBy → aggregations

Level 3 — Advanced JSON

arrays + structs + explode + window functions + deduplication + complex transformations

After this batch exercise, the natural next exercise is a **harder nested JSON dataset** containing arrays of products/items, customer structs, missing fields, duplicate orders, and malformed/optional data. That will prepare you very well for the Structured Streaming work you've been practicing.

